

Data Mining Complete Exam Notes

CT 72502 (Elective I) · BEI / BEX IV-I · TU, IOE · tiers over 17 papers, 2070–2081 BS

3. Classification

12 hrs · in 17/17 papers · 7 topics, 3 TOP tier · largest chapter in the syllabus · numericals → Ch 3 companion.

Weight of the chapter:

- Decision Tree Classifier **TOP 9/17** → nearly always a full numerical.
- Bayesian Classifier **TOP 8/17**
- Issues: Overfitting, Validation, Model Comparison **TOP 7/17** → **confusion-matrix numerical appears in 8 papers.**
- ANN Classifier **HOT 5/17** · Rule Based **HOT 4/17**
- Basics **PIN 3/17** · Nearest Neighbor **PIN 2/17**

3.1 Basics and Algorithms

Why is classification a supervised learning method? **2 10**

PIN 3/17 71 Shr, 74 Ch, 80 Bh

Classification

- Predicts **categorical** class labels (discrete, nominal).
- **Supervised**: the training tuples come w/ a known **class label**. The model learns from labelled answers.
- vs **unsupervised** (clustering): no labels given, groups discovered. → Ch 5
- vs **prediction / regression**: predicts a *continuous* value, not a label.

2-step process (**this is the block diagram Q**)

1. **Learning / model construction** (training)
 - Input: **training set**, each tuple w/ its class label.
 - Output: the model, as rules / tree / formula.
2. **Classification / model usage** (testing then deployment)
 - Estimate accuracy on an **independent test set**.
 - **Test set must be separate from the training set**, else the accuracy is optimistic (the model has memorised).
 - If accuracy acceptable → classify new, unlabelled data.

Where each classifier fits

- **Eager learners** build the model *before* seeing a query: decision tree, Bayes, ANN, rule-based, SVM.
- **Lazy learners** store the data and act only at query time: KNN, case-based.

Applications

- Credit / loan approval, fraud detection, medical diagnosis, target marketing, churn prediction, spam filtering.

Answer skeleton **10**

1. Definition + “supervised because labels are given”.
2. **Draw the 2-step block diagram** (training → model → testing → deployment).
3. Name the classifiers, 1 line each.
4. Close w/ the 2–3 metrics used to judge one.

3.2 Decision Tree Classifier

What is a decision tree? How is information gain used for attribute selection? 8 10 4+6

TOP 9/17 79 Bh, 78 Bh, 76 Ch, 76 Ash, 75 Ch, 74 Ch, 73 Shr, 72 Ka, 71 Ch **Almost always a full tree-building numerical** → companion.

Structure

- Flow-chart-like **inverted tree**.
- **Root node**: topmost, the first test.
- **Internal node**: a test on one attribute.
- **Branch**: an outcome of that test.
- **Leaf node**: holds a **class label**.
- Path root → leaf = one classification rule.

Why popular

- No domain knowledge or parameter setting needed.
- Handles high-dimensional data.
- **Human-readable** → easy to convert to IF-THEN rules.
- Fast; good accuracy in most cases.

Hunt's algorithm 72 Ka 10 → the ancestor of all of them

Let D_t = training records reaching node t , $y = \{y_1 \dots y_c\}$ the class labels.

1. If all records in D_t belong to the same class $y_t \rightarrow t$ is a **leaf** labelled y_t .
 2. If D_t is empty $\rightarrow$ leaf labelled w/ the **majority class of the parent**.
 3. If D_t has records of more than 1 class $\rightarrow$ pick an **attribute test**, split D_t into subsets, **recurse** on each subset.
- Descendants: **ID3** → **C4.5** → C5.0 (Quinlan), **CART**, SLIQ, SPRINT.

Generic induction algorithm

- **Greedy, top-down, recursive, divide-and-conquer**.
- Greedy = picks the locally best attribute at each node, never backtracks.
- **Stopping conditions**:
 - all tuples at the node share 1 class, or
 - no attributes left $\rightarrow$ leaf = majority class, or
 - no tuples left $\rightarrow$ leaf = majority class of parent.

8.2 Decision Tree Induction 331

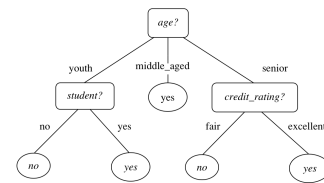


Figure 8.2 A decision tree for the concept *buys.computer*, indicating whether an *AllElectronics* customer is likely to purchase a computer. Each internal (nonleaf) node represents a test on

Test conditions per attribute type 72 Ka asks this

- **Binary**: 2 outcomes, forced.
- **Nominal**
 - *Multi-way*: 1 branch per value.
 - *Binary*: split values into 2 subsets. $2^{v-1} - 1$ ways.
- **Ordinal**: same as nominal, but a grouping must **not break the order**.
- **Continuous**
 - *Binary*: test $A \leq t$ vs $A > t$.
 - *Multi-way*: discretise into ranges first.
 - Split point: sort values, try the **midpoint** of each adjacent pair, $\frac{a_i + a_{i+1}}{2}$; v values $\rightarrow v - 1$ candidate splits. Pick the one w/ minimum $Info_A(D)$.

The 3 attribute selection measures (memorise all three)

1. Information gain ID3 maximise

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

$$Gain(A) = Info(D) - Info_A(D)$$

- $p_i = |C_{i,D}|/|D|$ = fraction of class i . $Info(D)$ = **entropy**, in bits.
- Pick the attribute w/ the **highest** $Gain$.
- **Bias**: prefers attrs w/ **many distinct values** (a unique ID gives $Info_A = 0$, so maximal gain, yet is useless).

2. Gain ratio C4.5 maximise

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \log_2\left(\frac{|D_j|}{|D|}\right)$$

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

- Normalises the gain → **fixes the many-values bias**.
- Unstable as $SplitInfo \rightarrow 0$; constrain $Gain$ to be at least the average.

Overfitting + pruning 75 Ch 4

- Tree grows branches that fit **noise and outliers** → poor accuracy on new data.
- **Pre-pruning** (early stopping)
 - Halt splitting at a node; make it a leaf w/ the majority class.
 - Stop if: gain below a threshold, too few tuples, purity high enough.
 - **Hard to pick the threshold** → too high underfits, too low overfits.
- **Post-pruning** (grow full, then cut) → **preferred**
 - Remove subtrees from a fully grown tree, replace w/ a leaf.
 - CART uses *cost complexity* pruning w/ a separate prune set.
 - C4.5 uses *pessimistic* pruning, no extra data needed.
- Other fixes: more training data, cross-validation, ensembles.

3. Gini index CART minimise

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2$$

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

- **Gini considers only BINARY splits** → for v values, try all $2^{v-1} - 1$ subset splits and keep the best.
- Pick the attribute w/ the **largest** $\Delta Gini$ (equivalently smallest $Gini_A$).
- **Bias**: toward attrs w/ many values, and toward equal-sized balanced partitions.

Reading the measures

- All 3 measure **impurity**. Pure node → $Info = 0$, $Gini = 0$.
- Max impurity: 2 classes 50/50 → $Info = 1$, $Gini = 0.5$.

Advantages

- Simple, fast to build and to apply.
- Self-explanatory, converts directly to rules.
- No normalisation or scaling needed.
- Handles both nominal and numeric attrs, and irrelevant attrs.

Disadvantages

- **Greedy** → locally optimal, not guaranteed globally optimal.
- Prone to **overfitting** without pruning.
- Unstable: a small data change can restructure the whole tree.
- Axis-parallel splits only → struggles w/ diagonal boundaries.
- Biased toward attrs w/ many values (unless gain ratio used).

3.3 Rule Based Classifier

How does a Rule Based Classifier work? Explain with example. 7 2+8 9

HOT 4/17 80 Bh, 76 Ch, 74 Ash, 70 Ch

Form

- Model = a set of **IF-THEN** rules.

-

IF (age = youth) AND (student = yes) THEN buys_computer =

- **Antecedent** (LHS, the condition) → **Consequent** (RHS, the class).
- A rule **covers** a tuple if the antecedent holds; the rule is then said to be *triggered*.

Rule quality measures 70 Ch asks Accuracy + Laplace

verified/added

- n_{covers} = tuples covered by R , $n_{correct}$ = those it labels correctly, $|D|$ = size of the data set, k = no. of classes.

$$Coverage(R) = \frac{n_{covers}}{|D|} \quad Accuracy(R) = \frac{n_{correct}}{n_{covers}}$$

$$Laplace(R) = \frac{n_{correct} + 1}{n_{covers} + k} \quad m\text{-estimate}(R) = \frac{n_{correct} + kp}{n_{covers} + k}$$

- **Why Laplace:** plain accuracy rates a rule covering 1 tuple correctly as 1.0, which is meaningless. Laplace pulls small-coverage rules toward the prior, so a rule must cover *enough* tuples to score well.
- Eg 1 tuple, 1 correct, 2 classes: $Accuracy = 1/1 = 1.0$ but $Laplace = 2/3 = 0.67$.

Rule extraction from a decision tree

- 1 rule per **root-to-leaf path**. Each split condition ANDed; leaf = consequent.
- Mutually exclusive + exhaustive by construction.
- Rules can then be **pruned** individually, which may break exclusivity → then class-based ordering is applied.

Conflict resolution 76 Ch 2+4

- **Why needed:** a tuple may trigger **several rules predicting different classes**, or **no rule at all**.
- **Size ordering** (rule antecedent size): the *most specific* rule wins, ie the one w/ the most attribute tests. Intuition: more conditions = more evidence.
- **Class-based ordering:** classes sorted by decreasing prevalence (or by misclassification cost). All rules of one class come as a block.
- **Rule-based ordering (decision list):** rules ranked by quality (accuracy, coverage, size) by the expert or the algorithm. **First rule that fires wins**.
- **No rule fires** → use a **default rule**, predicting the majority class of the tuples not covered by any rule. Always placed last.

Two useful properties

- **Mutually exclusive:** no 2 rules fire on the same tuple → no conflict possible.
- **Exhaustive:** every possible tuple is covered → no default rule needed.
- Rules extracted from a decision tree are **both** by construction.

Sequential covering (direct rule induction) 70 Ch asks CN2

verified/added

- Learns rules **directly from the data**, one class at a time.
- Algorithms: **CN2**, **RIPPER**, **AQ**, **FOIL**.
- Steps:
 1. Pick a class. Learn 1 rule that covers many of its tuples and few others.
 2. **Remove** the tuples that rule covers.
 3. Repeat on what remains until a stopping criterion is met.
- **CN2** specifically: general-to-specific beam search; starts from the empty antecedent and repeatedly appends the test that most improves the rule, scored by **entropy** and checked by a **likelihood-ratio significance test**. Produces an ordered rule list (decision list) + a default rule.

3.4 Nearest Neighbor Classifier

What is a Nearest Neighbor Classifier? Main issues? Propose a fix. 1+3+4 3+6

PIN 2/17 80 Ba, 80 Bh 80 Bh is a numerical → companion.

Idea

- **Lazy learner** / instance-based: stores all training tuples, builds **no model**.
- All work happens at query time.
- “Tell me who your neighbours are and I will tell you who you are.”

Algorithm (KNN)

1. Choose k and a distance measure (usually Euclidean).
 2. Compute the distance from the query tuple to **every** training tuple.
 3. Take the k smallest → the k nearest neighbours.
 4. **Classification**: assign the **majority class** among those k .
Prediction: take their **mean** value.
- **Normalise the attributes first**, else large-range attrs dominate the distance.
 - k odd for 2 classes → avoids ties.

Choosing k

- k too small → sensitive to **noise**, overfits.
- k too large → neighbourhood includes other classes, underfits.
- Pick by cross-validation; rule of thumb $k \approx \sqrt{n}$.

Issues (80 Ba asks exactly this)

- **Expensive at query time**: $O(n)$ distance computations per query, no model to shortcut it.
- **Curse of dimensionality**: distances lose meaning in high dims.
- **Scale sensitive**: needs normalisation.
- **Irrelevant attributes** distort distance equally w/ relevant ones.
- Needs the whole training set in memory.

Fixes / classifier to propose

- Speed: k-d trees, ball trees, partial distance, editing (drop redundant tuples).
- **Attribute weighting** or feature selection for irrelevant attrs.
- **Distance-weighted voting**: nearer neighbours count more, $w = 1/d^2$.
- **Propose an eager learner** instead: a **decision tree** or **Naive Bayes** builds the model once, then classifies in near-constant time, and ignores irrelevant attrs.

3.5 Bayesian Classifier

Explain Naive Bayesian classification with a suitable example. 8 6 10 3+7

TOP 8/17 81 Ba, 79 Bh, 78 Bh, 76 Ash, 75 Ash, 72 Ch, 71 Ch, 71 Shr **Numerical in 79 Bh and 72 Ch** → companion.

Bayes theorem

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}$$

- X = the tuple (evidence), H = hypothesis “ X belongs to class C ”.
- $P(H)$ = **prior**: probability of the class before seeing X .
- $P(H|X)$ = **posterior**: after seeing X . **This is what we want.**
- $P(X|H)$ = **likelihood**. $P(X)$ = **evidence**.
- 76 Ash asks prior vs posterior directly.

Naive Bayes

- Classify X to the class C_i w/ the largest $P(C_i|X)$.
- $P(X)$ is the **same for every class** → ignore it; just maximise $P(X|C_i)P(C_i)$.
- **“Naive” = class-conditional independence assumption**: given the class, the attributes are assumed independent, so

$$P(X|C_i) = \prod_{k=1}^n P(x_k|C_i)$$

- That turns 1 hard joint probability into n easy counts. **This is the whole trick.**

Algorithm steps (write these, then compute)

1. Compute the prior $P(C_i) = |C_{i,D}|/|D|$ for each class.
2. For each attribute value x_k in X , compute $P(x_k|C_i)$ from the counts.
 - Categorical: $P(x_k|C_i) = |\text{tuples in } C_i \text{ with } x_k|/|C_{i,D}|$.
 - Continuous: assume Gaussian, $g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$.
3. Multiply: $P(X|C_i) = \prod_k P(x_k|C_i)$.
4. Compute $P(X|C_i)P(C_i)$ for every class.
5. **Predict the class w/ the maximum value.**

Laplacian correction 78 Bh short note 4

- **Problem**: if any $P(x_k|C_i) = 0$, the whole product collapses to **zero**, wiping out all the other evidence.
- **Fix**: add 1 to every count (and k to each denominator, k = no. of values):

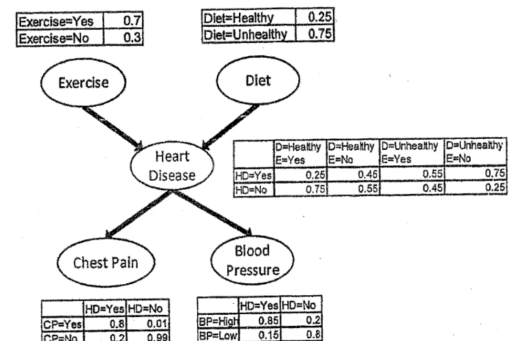
$$P(x_k|C_i) = \frac{n_k + 1}{|C_{i,D}| + k}$$

- Eg counts 0, 990, 10 → probabilities 0, 0.990, 0.010; after correction $\frac{1}{1003}, \frac{991}{1003}, \frac{11}{1003} = 0.001, 0.988, 0.011$.
- **Estimates barely move, but the zero is gone.**

Bayesian Belief Network (BBN) 3+5 10

81 Ba, 71 Ch 81 Ba is a numerical → companion.

- **Limitation of Naive Bayes**: the independence assumption is **almost never true** in practice. Correlated attributes get double-counted, which skews the posterior.
- **BBN drops it**: allows a *subset* of attributes to be conditionally dependent.
- Two parts:
 1. **Directed acyclic graph (DAG)**: node = variable, arc = direct causal dependence. Arc $Y \rightarrow Z$ means Y is a **parent** of Z .
 2. **Conditional probability table (CPT)** at each node, giving $P(Z | \text{parents}(Z))$.
- Joint probability: $P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(x_i))$
- A node is conditionally independent of its non-descendants given its parents.
- **When to use which** 71 Ch:
 - **Naive Bayes** → attributes roughly independent, need speed, little data, many attributes. Eg text/spam classification.
 - **BBN** → known causal structure and clear dependencies, need to reason w/ missing values. Eg medical diagnosis.



The 81 Ba network and its CPTs. Solved in the companion.

Advantages

- Fast to train and to classify; single pass over the data.
- Needs little training data.
- Handles missing values naturally (skip that factor).
- **Provably optimal** when the independence assumption actually holds.
- Incremental: counts can be updated as data arrives.

Disadvantages

- Independence assumption rarely holds → accuracy loss on correlated attrs.
- Zero-frequency problem (needs Laplacian correction).
- Cannot model interactions between attributes.
- Continuous attributes need a distribution assumption (usually Gaussian) or discretisation.

3.6 Artificial Neural Network Classifier

How does an ANN classifier work? Explain with algorithm. 8 2+6 5

HOT 5/17 81 Ba, 80 Ba, 76 Ch, 75 Ch, 72 Ka 76 Ch is a gradient numerical → companion.

Structure

- Set of connected **units (neurons)** in layers, each connection carrying a **weight**.
- **Input layer** (1 unit per attribute) → one or more **hidden layers** → **output layer** (1 unit per class).
- **Multilayer feed-forward**: no cycles, no connections within a layer.
- Each unit: net input = $\sum_i w_i x_i + \theta$ (bias), then an **activation function**.
- **Sigmoid (logistic)**:

$$\sigma(\theta) = \frac{1}{1 + e^{-\theta}}, \quad \sigma'(\theta) = \sigma(\theta)(1 - \sigma(\theta))$$

- **That derivative identity is why sigmoid is used** → makes backprop cheap.

Backpropagation algorithm

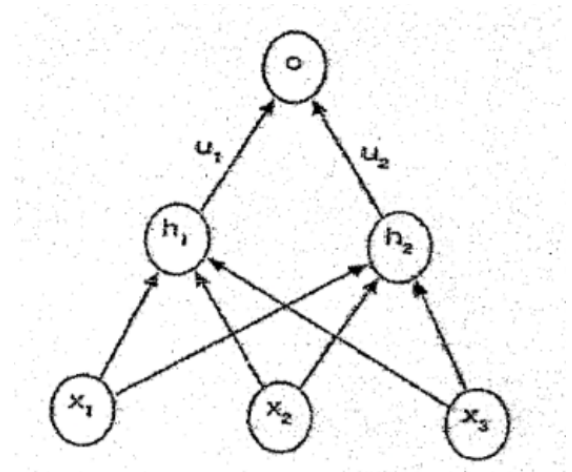
1. **Initialise** weights and biases to small random numbers.
2. **Forward pass**: compute net input and output for each unit, layer by layer.
3. **Compute error at the output**: $Err_j = O_j(1 - O_j)(T_j - O_j)$ where $T_j = \text{target}$.
4. **Backpropagate**: for a hidden unit, $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$.
5. **Update weights and biases**: $w_{ij} += l \cdot Err_j \cdot O_i$, $\theta_j += l \cdot Err_j$ (l = learning rate).
6. **Repeat** until the terminating condition (error small, or epoch limit).

Advantages

- High tolerance of **noisy** data.
- Classifies patterns it was not trained on.
- Works when the relationship is unknown / non-linear.
- Well suited to continuous-valued inputs and outputs.
- Inherently **parallel**.

Disadvantages

- **Black box**: poor interpretability, hard to justify a decision.
- Long training times.
- Many parameters set empirically (topology, l , epochs).
- Needs numeric input → encoding effort.
- Can get stuck in a **local minimum**.



The 76 Ch network. Gradients derived in the companion.

General considerations 72 Ka asks this

- **Network topology**: no. of hidden layers and units. Chosen by trial and error; no rule.
- **Input encoding**: normalise inputs to $[0, 1]$; one unit per nominal value.
- **Output encoding**: 1 unit for 2 classes, else 1 per class.
- **Learning rate l** : too large → oscillation; too small → slow. Often $1/t$ w/ epoch t .
- **Initial weights**: small random, to break symmetry.
- **Update mode**: case updating (per tuple) vs epoch updating (per pass).

3.7 Issues: Overfitting, Validation, Model Comparison

Confusion matrix: calculate accuracy, sensitivity, specificity, precision, recall. 4 10 3+5

TOP 7/17 80 Ba, 78 Bh, 75 Ash, 74 Ash, 73 Shr, 72 Ch, 70 Ch, 74 Ch **8 papers. Highest-yield numerical in the whole syllabus** → companion.

The 4 building blocks

- **TP**: positive tuples correctly labelled positive.
- **TN**: negative tuples correctly labelled negative.
- **FP**: negative tuples wrongly labelled positive (*false alarm*).
- **FN**: positive tuples wrongly labelled negative (*miss*).
- $P = TP + FN$ (all actual positives), $N = FP + TN$ (all actual negatives).

		Predicted class		Total
		yes	no	
Actual class	yes	TP	FN	P
	no	FP	TN	N
Total		P'	N'	P + N

All the formulas (one table, learn it whole)

Measure	Formula
Accuracy	$(TP + TN)/(P + N)$
Error rate	$(FP + FN)/(P + N) = 1 - \text{accuracy}$
Sensitivity, recall, TPR	TP/P
Specificity, TNR	TN/N
False alarm rate, FPR	$FP/N = 1 - \text{specificity}$
Precision	$TP/(TP + FP)$
F_1 score	$\frac{2 \times \text{prec} \times \text{rec}}{\text{prec} + \text{rec}}$
F_β	$\frac{(1 + \beta^2) \text{prec} \times \text{rec}}{\beta^2 \text{prec} + \text{rec}}$

- **Recall = sensitivity = TPR**. Three names, one thing. Papers use all three.
- F_1 = harmonic mean of precision and recall.

When accuracy is the wrong measure 80 Ba 3

- **Class imbalance**. If 99% of tuples are negative, a classifier that always says “negative” scores **99% accuracy** and is useless.
- Examples: cancer screening, fraud detection, intrusion detection, defect detection.
- **Use instead**: sensitivity + specificity separately, precision/recall, F_1 , ROC/AUC, or cost-sensitive measures.
- Note: $\text{accuracy} = \text{sensitivity} \frac{P}{P+N} + \text{specificity} \frac{N}{P+N} \rightarrow$ accuracy hides the split.

Precision vs recall trade-off 74 Ch asks the inverse relation

- Loosen the threshold → more positives flagged → recall ↑, precision ↓.
- Tighten it → precision ↑, recall ↓.
- Perfect precision *and* recall is only possible w/ a perfect classifier → report F_1 to balance them.

THE trap in this chapter. Han & Kamber put **actual on the rows, predicted on the columns**. The IOE papers flip this between years: 74 Ch and 75 Ash write “Actual \ Predicted”, but 78 Bh and 72 Ch write “Predicted / Actual”.

If you read the orientation wrong, **FP and FN swap**, so sensitivity and specificity swap and precision is wrong. **Always copy the row/column labels onto your answer sheet first**, mark which axis is actual, then compute.

verified/added

Validation methods

- **Holdout**: split, typically $\frac{2}{3}$ train / $\frac{1}{3}$ test. Fast, but depends on the one split.
- **Random subsampling**: holdout repeated k times, accuracies averaged.
- **k -fold cross-validation (the standard)**
 - Split into k equal folds. Train on $k - 1$, test on 1. Rotate k times.
 - Every tuple is used for training $k - 1$ times and testing exactly once.
 - $k = 10$ **recommended** (low bias and variance).
 - *Stratified*: each fold keeps the class distribution. Preferred.
- **Leave-one-out**: $k = n$. Expensive; for very small data sets.
- **Bootstrap (.632)**: sample n tuples *with* replacement. On average 63.2% of tuples appear in the training set. Best for small data sets.

ROC curve 73 Shr 1

- Plot of **TPR (y) vs FPR (x)** as the decision threshold sweeps.
- Diagonal = random guessing. Top-left corner = perfect.
- **AUC** = area under the curve. Closer to 1 = better; 0.5 = useless.
- Lets you compare 2 classifiers **independently of the threshold** and of class balance.

Improving accuracy: ensembles (1 line each is enough)

- **Bagging**: train k models on bootstrap samples, take a **majority vote**. Reduces *variance*; helps unstable learners like decision trees.
- **Boosting / AdaBoost**: train models **sequentially**, each re-weighting the tuples the previous one got wrong; weighted vote. Reduces *bias*; can overfit noise.
- **Random forest**: bagging + a random attribute subset at each split. Robust to overfitting, handles many attributes.

3.8 PYQ Mapping

N marks a question fully solved in the Ch 3 Numerical companion.

3.1 Basics

1. PIN 3/17 Block diagram of classification stages + inverse precision/recall relation + confusion matrix N. 2+3+5 74 Ch
2. Why is classification supervised? + impurity measures. 10 71 Shr
3. When do we use a classifier? (asked w/ the KNN numerical) 3 80 Bh

3.2 Decision Tree

4. N Draw a decision tree using ID3. 10 79 Bh
5. N Construct a tree using **information gain** + predict a new tuple. 8 78 Bh
6. N Which attribute would ID-3 choose as root? + draw the full tree. 2+5 76 Ch
7. TOP /17 Decision tree + how information gain selects attributes, w/ example. 8 73 Shr
8. Accuracy of classifiers + how to select the best root attribute. 4+6 76 Ash
9. PIN 2/17 Gini-Index w/ example + handling overfitting. 6+4 75 Ch 10 71 Ch
10. PIN 1/17 **Hunt's Algorithm** + test conditions per attribute type. 10 72 Ka
11. PIN 1/17 Decision tree w/ the concept of Naive Bayes classification. 10 74 Ch

3.3 Rule Based

12. HOT 4/17 How a rule-based classifier works, w/ example. 7 80 Bh 2+8 74 Ash
13. PIN 1/17 Rule-based classifier + **CN2** + define **Accuracy** and **Laplace** measures. 9 70 Ch
14. PIN 1/17 Why is conflict resolution necessary? + common strategies. 2+4 76 Ch

3.4 Nearest Neighbor

15. N Classify a flower w/ **KNN**, $K = 3$, Euclidean. 3+6 80 Bh
16. PIN 1/17 What is KNN? Main issues? Propose a classifier that solves them. 1+3+4 80 Ba

3.5 Bayesian

17. TOP /17 Naive Bayes w/ suitable example. 8 75 Ash 6 78 Bh
18. N Classify $X = (\text{Home Owner}=\text{No}, \text{Marital Status}=\text{Married}, \text{Income}=\$120\text{K})$. 6 79 Bh
19. N Predict class for $X = (\text{age}=\text{youth}, \text{income}=\text{medium}, \text{student}=\text{yes}, \text{credit}=\text{fair})$. 10 72 Ch
20. PIN 1/17 Prior and posterior probabilities + algorithmic steps + strengths. 3+7 76 Ash
21. PIN 1/17 Bayes classifier + when to use Naive Bayes vs BBN. 10 71 Ch
22. PIN 1/17 Naive Bayes + solving overfitting in classification. 10 71 Shr
23. N Limitation of Naive Bayes + how BBN overcomes it + heart-disease inference. 3+5 81 Ba
24. PIN 1/17 Short note: **Laplacian correction**. 4 78 Bh

3.6 ANN

25. HOT 3/17 How does an ANN classifier work? w/ algorithm. 8 81 Ba, 75 Ch 2+6 72 Ka
26. N Compute activation at h_1 , $\partial E / \partial o$, $\partial E / \partial w_{12}$. 2+3+4 76 Ch
27. PIN 1/17 Short note: Neural Network Classifier. 5 80 Ba

3.7 Issues: Overfitting, Validation, Model Comparison

28. **N TOP 7/17** Confusion matrix numericals (8 papers, same method, different tables):

- Classification error, sensitivity, false alarm rate, specificity 3+5 80 Ba
- Accuracy, TPR, FPR, precision 4 78 Bh
- Accuracy, sensitivity, specificity, precision 10 75 Ash, 74 Ash
- + recall 10 72 Ch
- ROC + TPR, FPR, precision 1+3+6 73 Shr
- Build the matrix from 2 predicted sequences, then 4 metrics 10 70 Ch
- Given matrix → accuracy, sensitivity, precision 5 74 Ch

29. **PIN 1/17** When can you not use accuracy? Give examples. 3 80 Ba

30. **PIN 1/17** Short note: Overfitting problem in classification. 4 78 Bh

Coverage check

- All Ch 3 PYQs covered. **12 are numerical** → all solved in the companion.
- **Gaps filled by research** verified/added :
 - **Laplace and m-estimate** rule measures and the **CN2** algorithm 70 Ch: absent from every source note; taken from Tan/Steinbach and Han & Kamber §8.4.
 - **Conflict resolution strategies** 76 Ch: only named in the notes, not explained.
 - The **confusion-matrix orientation trap** (papers flip actual/predicted between years).

Sources used for this chapter

- **Han & Kamber 3rd ed**, ch 8 (basic concepts, decision tree induction, attribute selection measures, Bayes, rule-based, model evaluation) and ch 9 (BBN, backpropagation, lazy learners). **Primary authority; every formula above checked against it.**
- **DBK Lectures 5–11** (Thapathali): Hunt's algorithm, tree construction, Naive Bayes, KNN, ANN, evaluation.
- data mining_/Chapter 3 - Classification.pdf, hunts algorithm.pdf, ClassificationDecisionTree.ppt, Chapter8_2.ppt, Bayes_Net.docx.

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
DM	data mining	DW	data warehouse
DB	database	DBMS	database management system
KDD	knowledge discovery in databases	DSS	decision support system
OLTP	on-line transaction processing	OLAP	on-line analytical processing
ROLAP	relational OLAP	MOLAP	multidimensional OLAP
HOLAP	hybrid OLAP	ETL	extract, transform, load
BI	business intelligence	ER	entity-relationship
attr(s)	attribute(s)	dim(s)	dimension(s)
multidim	multidimensional	txn(s)	transaction(s)
info	information	ops	operations
repo	repository	freq	frequent / frequency
config	configuration	no.	number
PCA	principal component analysis	SVD	singular value decomposition
SMC	simple matching coefficient	KNN	<i>k</i> -nearest neighbour
ANN	artificial neural network	POS	point of sale
FK	foreign key	UI	user interface
GIGO	garbage in, garbage out	w/, w/o	with, without
rm	remove	vs	versus
Eg	example / for example	+Fig	draw a figure
→	leads to / therefore	+	and

Exam-paper month codes (used in the year tags): Ba = Baishakh · Jth = Jestha · Asa = Ashar · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Mng = Mangsir · Po = Poush · Ma = Magh · Ch = Chaitra.

Tier chips: **TOP /17** = topic asked in 7 or more of the 17 papers · **HOT /17** = 4–6 papers · **PIN /17** = 1–3 papers but worth ≥8 marks. Mark chips

such as **2+5** are the marks that question actually carried. **verified/added** marks anything researched and added that was not in the source notes.